01 / The merged product

HEATCHECK - NBA RESEARCH + PREDICTION-MARKET EVIDENCE Version 1.0 | Specification and prompts | Three separate computers

Build Jone, with Heat Check as its adversarial research engine: a conversational NBA researcher that proposes hypotheses, tries to disprove them, measures how often its own methods fail, and compares supported NBA forecasts with read-only prediction-market quotes. Ambitious integration is the target; correctness decides which claims are available, not a feature-count promise.

Keep three distinct outputs: basketball research findings, pre-game predictive advantage over a baseline, and read-only NBA forecast/market-quote comparisons. A hot-hand finding is not automatically a game-winner model or a profitable trade. Jone names the Discoverer agent inside HeatCheck; Challenger names its adversarial partner. No UI or pitch may collapse these channels into a generic 'alpha found' badge.

The product loop is voice/text or attached evidence -> structured claim -> scope/power check -> frozen plan -> Discoverer -> Challenger -> deterministic Referee -> immutable evidence -> optional market comparison. The user can inspect every rejection and ask what evidence would change the conclusion. Market comparison displays evidence; it does not find or rank trading opportunities.

The source plans contribute different strengths. Original Jone supplies the research pipeline, point-in-time discipline, formula proposals and complete ledger. Heat Check supplies planted truth, attack registry, shrinkage/reliability views, blind challenges, calibration diagnostics and the graveyard. This pack adds NBA event matching and quote semantics and the voice/page/file experience. Their long-term calendars and old human/team assignments do not govern this build.

Primary sponsor fit: the supplied Brevan Howard brief explicitly includes sports and asks for ideas tested against luck. incident.io additionally requires actual live-feed reactions, not merely voice commands. Marshall Wace needs a specific new-data/idea story. Capital One would require a genuinely different accessible-account-management objective. Do not assume multi-track entry is permitted or select a track solely from this document.

Read this document and the shared protocol first. Each Pro reads its own role document; deeper source material is available as reference. All examples, thresholds and dates below are proposed contracts until frozen against actual available data. No numerical result in this pack is a completed experiment.

02 / Merge decisions that prevent false confidence

Map all 97 Heat Check proposals in Heat_Check_97_Feature_Map.csv. CORE means a release target; STRONG means the next strengthening layer after core gates; CONDITIONAL means a stated dependency; LATER preserves research ambition without pretending completion. IDs allow exact task handoffs. Do not install 97 disconnected features or substitute a feature count for a working loop.

Adopt the synthetic harness, hidden truth, held-out generator family, complete candidate ledger and mandatory attack registration. Keep a measured power curve beside false-discovery estimates: rejecting everything is not a useful researcher. Fit, tune, calibrate and test on disjoint league IDs; final family and final NBA outcomes have separate access policies.

Change Heat Check's 'Real / Lucky / Underpowered' labels to SURVIVED_PROTOCOL / NOT_SUPPORTED / INSUFFICIENT_EVIDENCE. Keep DATA_INVALID and BLOCKED separate. Non-significance does not prove a finding was luck; all attacks passing does not prove truth. A simulated support score must name its generator distribution and cannot become '90% chance this NBA discovery is real'.

Correct important traps: post-shot assisted labels can leak made/missed outcomes; observed score margin must be reconstructed before the event; the simulator's probability-point effects cannot be inserted as logit coefficients. Permutation p-values can be discrete/conservative, so a mandatory KS-uniformity pass is inappropriate. A refinement counter or a small vault-query budget alone does not control adaptive testing.

Shot associations, game forecasts and market comparisons use separate estimands and schemas. Supporting a player's shooting effect cannot be converted to a team win probability without a separately evaluated bridge. NBA probabilities cannot price unrelated political or economic markets.

Keep the ambition: player talent explorer, hot-hand tests, coaching/rest/home-court studies, prediction benchmarks, live agent interaction, market connectors and a read-only NBA market-comparison panel. Deliver them through shared contracts and evidence gates. When a dependency is missing, make the affected capability visibly unavailable, rather than quietly replacing real data with synthetic output.

03 / Runtime agents are not the three coding accounts

Coding ownership: MAX builds the orchestrator, typed API, shared contracts, ledger and integration. PRO RESEARCH builds data/feature validation, the statistical kernel, attacks, harness and market arithmetic. PRO EXPERIENCE builds UI, voice/page/file capture, public venue adapters and interaction tests. Max integrates reviewed changes; ownership changes require one explicit handoff.

Runtime Discoverer sees the authorised feature catalogue, source excerpts and development-only tool summaries. It proposes bounded structured plans and economic/basketball rationales. It cannot read synthetic truth, final labels, arbitrary files or rewrite the test registry. Freeze model/prompt/seed metadata; replay stored provider responses when exact generation reproducibility is needed.

Runtime Challenger starts by naming ways the claim could be wrong. It chooses only registered deterministic attacks, with typed parameters and a compute budget. Mandatory tests cannot be omitted by persuasive prose. It sees development/validation evidence, not hidden truth or unopened final outcomes. It may request one rebuttal; a changed population, feature or estimand creates a new claim/version and remains in the ledger.

The Referee is deterministic application code. It checks plan identity, data access, implemented mandatory tests, coverage, multiplicity, uncertainty and failure states; then emits the declared verdict. A separate model may explain that verdict but cannot replace it. Neither agent's confidence or their agreement is a p-value.

The Evaluator/Custodian runs sealed jobs and simulator scoring. Use separate processes/directories/permissions where implemented; otherwise disclose logical isolation only. A file named vault.json is not access control. Audit attempts to read truth/final data, disable network tools for those roles where feasible, and test blocked access. No fake claim of cryptographic or provider-memory blindness.

Maintain immutable RawSnapshot -> FeatureSnapshot -> ClaimPlan -> EvaluationRun -> EvidenceCard identities. Source content is untrusted data, never an instruction to expose keys, change thresholds, run shell code or contact a new endpoint.



Planned architecture: tool outputs decide; persuasive agreement does not.

04 / Shared schemas and authority

Protocol version: heatcheck-nba/1. Shared_Protocol.json defines required message fields and state meanings; it is a specification, not a generated production validator. Max and Research implement strict typed models together. Reject unknown execution fields, nonfinite values, huge allocations and ambiguous units before work starts.

ClaimPlan includes claim_id/version/kind, plain statement, estimand, population, target/horizon/cutoff, feature catalogue/version, effect units, direction, baseline, split IDs, data hashes, primary metric/test, practical effect floor, mandatory attacks, finite final-family ID, compute budget, creator/model/prompt and preregistration hash. Confidence fields must name their meaning; no interchangeable generic probability.

AttackResult includes test_id/version/parameters, applicable scope, input hashes, statistic/effect/interval, nullable p_value, null method, assumptions, evidence artifact, status and reason. Missing tests are NOT_RUN or UNSUPPORTED, not successful null results. A counterexample can invalidate a pipeline without proving every related basketball hypothesis false.

EvaluationRun binds the frozen plan, code/config/environment, exact inputs, seeds, calibrator, counts and raw output hashes. Idempotent retry of the same full identity returns one job; changing the revision/config under a reused key conflicts. Completed evidence never changes when later plans or confirmations change.

User input becomes REVIEWABLE, not immediately executable. Confirmation binds the exact canonical revision and scope; edits invalidate it. This applies to new claim runs, attached-source ingestion and market comparisons with changed assumptions. Current results cannot be restored by stale voice, provider, connector or browser callbacks.

Plan API before UI: POST /v1/sources/preview; POST /v1/claims/validate; POST /v1/claims/{id}/confirm; POST /v1/runs; GET /v1/runs/{id}; GET /v1/evidence/{id}; POST /v1/comparisons/validate and /confirm; POST /v1/comparisons. Define owner/context scope even for a local demo; a client-supplied owner header is not public authentication. No order, deposit, wallet-signing or withdrawal route belongs in this build.

05 / Hypothesis and model-search workflow

Use a small allowlisted plan language, never model-written executable Python. Start with game-level rest/home/strength hypotheses and shot-level hot-hand/clutch templates, each with a named estimand. Do not allow an arbitrary statement such as 'this player is clutch' to select its own convenient denominator, time window or target after results are seen.

Development can explore and learn from its own failures. Log every proposal, duplicate, invalid plan, refinement, tested variant and abandoned run. Register the validation/final plan after development and before protected evaluation; call it that, not pre-registration before any exploration. Hash the exact canonical bytes, versions and artifact references.

For a finite final family, register at most five selected hypotheses with complete endpoints before unlocking outcomes. Use separate statistical procedures for observational shot claims and predictive-score comparisons. Holm is the proposed fixed-family confirmation policy; Research must validate the underlying p-values. Do not combine Holm, BH, LORD and e-values as if they were interchangeable labels.

Bind unlock to one frozen family and final_epoch_id. The custodian atomically reserves the underlying record/game-ID partition and marks exposure spent before final labels/scores leave it. A new family ID, renamed file or revised snapshot containing those records cannot buy a fresh confirmatory look. Exact-identity retry/replay is logged as such; it cannot reset exposure. Persist this registry across restarts.

Audit prior exposure. Public historical NBA results and pretrained model knowledge mean sealing a local file today does not necessarily make the past untouched. Report historical pseudo-out-of-sample when contamination is known or plausible. Only genuinely future games after a recorded lock can establish prospective evidence; do not invent that evidence during a one-day build.

Before final: freeze coverage/exclusions, baseline update rules, clipping epsilon, all preprocessing, candidate identities, attacks, calibrator and primary effect threshold. While final runs, neither model may adapt from labels or intermediate scores. Publish the whole family result internally, not only its winner. A repair or hypothesis change after exposure requires a new experiment and new untouched data for confirmatory claims.

Share a result only as SURVIVED_PROTOCOL with named limits. Model-relative market comparisons additionally need the exact event/contract match and current quote context; forecast success alone does not establish money-making alpha.

06 / Read-only NBA market comparison

Polymarket and Kalshi remain optional read-only context connectors. Fetch supported NBA event metadata and timestamped quotes; compare them with a separately evaluated game forecast. There is no general-market scanner, portfolio search, trade ranking, return calculator, position sizing or execution route. This scope correction supersedes the earlier draft.

Use three canonical channels: SHOT_ASSOCIATION, GAME_PREDICTION and MARKET_COMPARISON. A comparison card displays the frozen model probability, any justified interval, the matched contract's observed bid/ask/midpoint with clear quote type, source time, data age and event/rule caveats. A price difference is descriptive context, not proof of a profitable strategy.

Match exact NBA event, participants/date, regulation/overtime, cancellation/postponement, correction/dispute and payout convention before placing model probability beside a contract quote. Similar titles or LLM semantic similarity only propose mappings. Ambiguous mappings are BLOCKED_RULES_UNVERIFIED. NBA models are not used for unrelated prediction markets.

Experience owns actual public adapters; Research validates probability/price units and metric arithmetic; Max owns typed event matching, authority and evidence. Store raw payload hashes, market/outcome IDs, adapter/API version, server/received/fetched times, unit conventions and mapping provenance. Never display midpoint or last trade as an executable ask. Missing/stale/paused data is a visible state.

No wallet keys, orders, deposits, withdrawals, account writes, automatic provider billing or geo-restriction bypass. Market-data access and hypothetical forecast comparisons do not establish trading eligibility, historical returns or real-time execution. No venue is claimed connected by this specification.

07 / Stack, providers and security boundaries

Target Python 3.12 with uv lock; NumPy/SciPy/pandas, scikit-learn and statsmodels for implemented kernels; SQLite for transactional revisions/jobs/ledger and Parquet for snapshots. FastAPI/Pydantic/Uvicorn expose a single backend. Experience may use a familiar React/Vite/TypeScript interface for rich live interaction, or plain web assets if stronger locally; freeze one choice at the first integration checkpoint. Do not mix Streamlit, a second SPA and a native shell.

Keep modules: heatcheck/agents, heatcheck/api, heatcheck/contracts, heatcheck/data, heatcheck/research, heatcheck/attacks, heatcheck/sim, heatcheck/markets/normalization, heatcheck/markets/adapters, heatcheck/ledger, apps/web, tests and reports. Pure statistical/comparison functions have no HTTP/provider/file side effects. Document files and skeletons are not implemented capabilities.

Provider plan: use the user's reported US$141 Meta credits where actual access permits. Muse Spark proposes/explains structured research; Muse Voice Transcribe supplies speech input; browser TTS is the initial output path. Groq is an optional rate-limited fallback, not Grok or an unlimited entitlement. Meta's documentation describes reasoning and speech endpoints; inspect the actual account and intended host before relying on them [S10-S12].

Keys stay server-side in private configuration; coding sessions must not print secret files. No automatic paid-plan signup, Contributor/training-data opt-in, API top-up or new provider account. Apply per-job tokens/audio/time/retry budgets, cache provider responses with exact prompt/source/model identity, and preserve a demo reserve. Meta credits do not pay other hosts' bills.

Runtime tool permissions are allowlisted: registered tests, authorised source handles and public GET connectors. No arbitrary URL fetcher, unrestricted SQL, shell execution or model-generated imports. Defend against document/page injection, CSV formula export, HTML injection, oversized/decompression-bomb files and path traversal. Test failure visibility, retry/backpressure, cancellation and stale responses; never mask outage with invented data.

08 / Three-computer delivery and the 24-hour pipeline

Latest staffing is three AI coding sessions on different computers, not the earlier two-account/same-PC arrangement. They need the same exact protocol and source baseline. Assign one branch per owner and one integration owner; no simultaneous shared-schema edits. Remote messaging between three separate accounts is not established by local /list-agents support.

Use an authorised shared repository/task channel with source-linked packets. A task is READY -> CLAIMED(owner,base) -> PATCH_READY(checks,artifacts) -> REVIEW_REQUIRED -> ACCEPTED or REPAIR_REQUIRED -> INTEGRATED. Git records/synchronises code; it does not wake another model, run an unconfigured CI or judge correctness. A dispatcher/polling workflow is a separate setup requiring actual access and a delivery acknowledgement. No external channel, repository push or credentials were configured by this pack.

Preserve the user's prior no-push restriction until explicitly released. Before then exchange local Git bundles/patches through an authorised shared medium. Do not reset preparation dates or claim old material was event-built. Confirm organisers' rules for pre-event specifications, code, data and AI use before dependent entry reuse.

Suggested windows from the actual start: H0-1 freeze contract, verify data slice/provider/capture and source transport; H1-5 build real source -> plan -> attack -> referee -> evidence plus connector normalisation in parallel; H5-9 integrate synthetic truth, actual NBA claim and visible failed/unsupported paths; H9-14 run harness and held-out evaluation, add game forecast and read-only market comparison; H14-18 independent replay, event-mapping/quote-unit counterexamples and voice workflow; H18 feature freeze; H20 release freeze; H20-23 rehearse/export/submit, reserving the final hour for genuine access/upload failures.

These are proposed coordination checkpoints, not a reason to assume impossibility or to fake completion. Ambition includes the full core and rich interaction. Cut unverified claims/optional polish first; correctness, negative controls and truthful provenance remain.

09 / Acceptance and progressive build discipline

Use Acceptance_Framework.json as the proposed gate ledger. Every gate starts NOT_RUN. Close it with exact code/data/config/seed, raw outputs, environment and a reviewer other than the author. Reviewer role independence is not a claim of independent human validation or statistical independence between models.

Required evidence: actual data validation and timestamps; tiny statistical/quote-unit oracles; null and planted-signal tests; correct discrete-p-value behaviour; train/calibration/final separation; finite strict JSON; no truth/vault access by agents; held-out family diagnostics; no auto-transfer of simulated trust to NBA; same-game forecasting comparison including fallbacks; NBA event mapping; quote-side/unit/freshness checks; actual UI/voice correction/export; source injection and resource/error cases.

Max independently reviews Research's kernels and Experience's authority/race/capture tests. Research reviews Max's DSL, registry/gates, ledger identity and final-data boundary. Experience runs actual integration against real backend responses and contributes counterexamples; author-written UI tests do not themselves constitute separate acceptance of that UI.

The agent must not stop at a plan, stub, green screenshot or count of tests. Per bounded task: derive expectations -> implement -> execute -> inspect actual outputs -> repair root causes -> rerun affected checks -> separate review -> checkpoint. Do not weaken expectations, hard-code answers, silently omit mandatory tests, invent latencies or relabel historical evidence as current.

Checkpoint task/source/protocol, changed files, real commands/results, NOT_RUN gates, defects and next receiver. Keep routine packets concise and reserve a full milestone report for release. If stuck, identify the exact missing dependency, continue independent authorised work, and send one actionable question. Do not burn budget rereading all packs or running conversational agent loops without new evidence.

A pipeline may be software-accepted while research remains inconclusive. A live connector may work while no comparable supported NBA market is available. A statistic may be computed correctly while its scientific claim lacks power. Keep these dispositions separate in code, UI and pitch.

10 / Max bootstrap and reference map

Copy this as the first instruction: 'You are HeatCheck-Max, the integration lead for the merged Jone/Heat Check NBA system. Read this role document, Shared_Protocol.json, Acceptance_Framework.json and the feature map. The latest setup is one Max and two Pro sessions on three computers. Older BreakPoint/MSM, two-session and same-PC assignments are superseded for this proposed product. Inspect actual code, tools and authorisation; no app/data/provider test has been completed merely because this pack exists.'

Continue: 'Agree one protocol/version and valid/invalid fixture with Research and Experience. Own orchestration/API/ledger/integration, not your teammates' paths. Build the real Discoverer -> Challenger -> deterministic Referee -> immutable evidence loop. Keep the shot, game-prediction, simulated-trust and market-comparison channels separate. Use only registered tools; never let an LLM compute or override a numerical verdict. Test a rejected null and a detectable planted effect. Treat claims/calibration/market comparisons as unvalidated until their gates are executed.'

Continue: 'Work autonomously through authorised reversible implementation/checks/repairs. Send exact source-linked review packets; preserve stable candidates. Obtain separate review before milestone acceptance. Maintain actual PASS/FAIL/NOT_RUN and hidden-data exposure limits. No orders, wallet writes, billing activation, publishing or outside messages from this document. Honour current user permissions and event rules. At each checkpoint report actual behaviour, evidence, defects and next smallest task; do not stop at a promise or fabricate a pass.'

Sources (checked 7 November 2026): [S1] user Jone_Project_Plan.pdf (20 pages); [S2] user heat-check-plan.md.pdf (27 pages), interpreted and corrected here; [S3] shufinskiy NBA data, github.com/shufinskiy/nba_data; [S4] swar/nba_api; [S5] docs.balldontlie.io; [S6] scikit-learn model evaluation and calibration docs; [S7] statsmodels multipletests; [S8] Miller/Sanjurjo 2018, doi:10.3982/ECTA14943; [S9] Google Research reusable-holdout article; [S10] dev.meta.ai/docs/overview; [S11] dev.meta.ai/docs/speech-to-text; [S12] console.groq.com/docs/rate-limits; [S13-S17] Polymarket/Kalshi market-data, orderbook, fixed-point and fee documentation. Sources.md contains exact links. Existence checks are not API execution, licensing approval or security audits.

Verified primary sources

Checked 7 November 2026. References support methods/API contracts; they do not establish successful calls, permissions, real opportunities or executed acceptance. User plans were merged with the corrections stated in this document.

[S1] User source: Jone_Project_Plan.pdf User-provided 20-page plan, 14 October 2026

[S2] User source: heat-check-plan.md.pdf User-provided 27-page Heat Check feature/research plan

[S3] Bulk NBA data repository https://github.com/shufinskiy/nba_data

[S6] Forecast scoring / calibration https://scikit-learn.org/stable/modules/model_evaluation.html

[S7] Multiplicity implementations
https://www.statsmodels.org/stable/generated/statsmodels.stats.multitest.multipletests.html

[S8] Miller and Sanjurjo (2018), finite-sequence bias
https://rua.ua.es/dspace/bitstream/10045/85611/1/2018_Miller_Sanjurjo_Econometrica.pdf

[S9] Adaptive analysis / reusable holdout, author explanation
https://research.google/blog/the-reusable-holdout-preserving-validity-in-adaptive-data-analysis/

[S10] Meta Model API overview https://dev.meta.ai/docs/overview

[S11] Meta speech transcription https://dev.meta.ai/docs/speech-to-text

[S12] Groq rate limits https://console.groq.com/docs/rate-limits

[S13] Polymarket public market data https://docs.polymarket.com/market-data/overview

[S14] Kalshi public market data https://docs.kalshi.com/getting_started/quick_start_market_data

[S16] Polymarket fees https://docs.polymarket.com/trading/fees

[S17] Kalshi current fee schedule https://kalshi.com/fee-schedule